

Week 1 Intern Plan: Foundations For Building AI Agents

Purpose

Welcome to your first-week exercise on building foundations for AI agents.

This week is designed to help you understand the core building blocks behind practical AI applications: prompt engineering, invoking LLM APIs, retrieval-augmented generation, and context engineering. By the end of the week, you should be able to explain these ideas clearly and apply them to a small assistant workflow.

This is a learning exercise, not a production assignment. Prioritize clear thinking, simple working examples, careful notes, and honest reflection over sophisticated infrastructure.

End-Of-Week Outcome

By the end of the week, you should be able to:

- * Explain how LLM applications differ from simple chatbot interactions.
- * Write prompts that include clear instructions, context, constraints, and output formats.
- * Make or describe a basic LLM API call.
- * Request and validate structured model output.
- * Explain how retrieval-augmented generation grounds responses in external knowledge.
- * Design context for a multi-step assistant workflow.
- * Present a small assistant prototype or design with risks, human review points, and evaluation criteria.

Week Overview

Day	Theme	Main Outcome
Day 1	LLM mental models and prompt engineering	Improved prompts and a reflection on when prompting is enough
Day 2	Invoking LLM APIs	Basic API call, structured output, and simple validation
Day 3	Retrieval-augmented generation	Grounded answers using retrieved context and citations
Day 4	Context engineering and workflow design	Context package for a multi-step assistant workflow
Day 5	Mini capstone and presentation	Small assistant prototype or design walkthrough

Suggested Setup

You should have:

- * A laptop with Python or JavaScript available.
- * A text editor or notebook environment.
- * Access to an LLM through an API, web interface, or local model.
- * A folder where you can save daily notes, prompt examples, small scripts, and your final capstone.

If you do not have API access, you can still complete the week by writing request/response pseudocode, manually testing prompts in a web interface, and validating sample JSON locally.

Day 1: LLM Mental Models And Prompt Engineering

Goal

Build intuition for how prompts shape model behavior and where prompting fits in a larger AI application.

Learn

Study these concepts:

- * What an LLM is and what it does.
- * Tokens, context windows, instructions, and outputs.
- * The difference between a chatbot, assistant, automation, and agent.
- * The basic LLM application pattern: prompt + model + data + tools + context.
- * Prompt anatomy: role, task, context, constraints, examples, and output format.

- * Zero-shot, one-shot, and few-shot prompting.
- * Why useful generative AI is not automatically an agent.

Public learning material:

- * OpenAI Prompting Guide: <https://platform.openai.com/docs/guides/prompting>
- * OpenAI Text Generation Guide: <https://platform.openai.com/docs/guides/text-generation>
- * OpenAI Cookbook: <https://github.com/openai/openai-cookbook>

Exercise

Create prompts for four tasks:

- * Summarize a long document for a project lead.
- * Extract action items from meeting notes.
- * Classify customer feedback into categories.
- * Produce structured output such as JSON or a table.

For at least three of the tasks, write:

- * A weak prompt.
- * An improved prompt.
- * A short explanation of what changed and why.

Your improved prompt should usually include:

- * The role or perspective the model should take.
- * The specific task.
- * Relevant context.
- * Constraints or rules.
- * Desired output format.
- * Instructions for uncertainty, missing information, or assumptions.

Daily Deliverables

- * Notes explaining what makes a prompt reliable.
- * Three before-and-after prompt examples.
- * A short reflection: "Where prompting helps, and where prompting alone is not enough."

Self-Check

You should be able to answer:

- * What makes a prompt ambiguous?
- * Why does output format matter?

- * When should you give examples?
- * What should the model do when information is missing?

Day 2: Invoking LLM APIs

Goal

Understand how application code calls an LLM and handles model responses.

Learn

Study these concepts:

- * Request structure: model, input/messages, instructions, temperature, max output tokens, and response format.
- * Non-streaming vs streaming responses.
- * Structured outputs and JSON schema.
- * Tool/function calling at a conceptual level.
- * Cost, latency, retries, rate limits, and error handling.
- * Why application code should validate model output before using it.

Public learning material:

- * OpenAI Responses API Reference: <https://platform.openai.com/docs/api-reference/responses>
- * OpenAI Structured Outputs Guide: <https://platform.openai.com/docs/guides/structured-outputs>
- * OpenAI Function Calling Guide: <https://platform.openai.com/docs/guides/function-calling>
- * OpenAI API Streaming Reference: <https://platform.openai.com/docs/api-reference/streaming>

Exercise

Create a small script, notebook, or pseudocode example that:

- * Sends a prompt to an LLM.
- * Requests structured JSON output.
- * Parses the response.
- * Validates required fields.
- * Handles invalid JSON or missing fields gracefully.

Suggested example task:

```
Input: A customer support message.  
Output JSON:  
{  
  "category": "billing | technical | account | other",  
  "urgency": "low | medium | high",  
  "summary": "...",  
  "missing_information": ["..."],
```

```
    "suggested_next_step": "..."  
}
```

If you cannot call an API directly, write the request body you would send, manually produce a sample response, and write validation code for that response.

Daily Deliverables

- * A basic LLM API call or request/response pseudocode.
- * A structured-output example.
- * Validation logic or a clear validation plan.
- * Notes on at least one failure case and how your code should handle it.

Self-Check

You should be able to answer:

- * What fields are required to make a basic model request?
- * Why is structured output useful?
- * What can go wrong when parsing model output?
- * Why should model output be treated as untrusted until validated?

Day 3: Retrieval-Augmented Generation

Goal

Learn why grounding matters and how retrieval changes answer quality.

Learn

Study these concepts:

- * Why prompts alone are not enough for private, current, or domain-specific knowledge.
- * Hallucination vs grounded answering.
- * The RAG pipeline: collect documents, chunk, embed, retrieve, generate.
- * Similarity search and top-k retrieval.
- * Chunk size, metadata, and citation tradeoffs.
- * Why citations and uncertainty improve reviewability.

Public learning material:

- * OpenAI Embeddings Guide: <https://platform.openai.com/docs/guides/embeddings>
- * Original RAG paper: <https://arxiv.org/abs/2005.11401>
- * Microsoft Learn RAG Overview:
<https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview>
- * LangChain RAG Tutorial: <https://docs.langchain.com/oss/python/langchain/rag>

Exercise

Create a small knowledge base using three short documents. You can write your own documents, such as:

- * Remote work policy.
- * Travel expense policy.
- * Customer data handling policy.

Then:

- * Break each document into smaller sections.
- * Ask three questions that require information from the documents.
- * Retrieve or manually select the most relevant sections for each question.
- * Answer each question using only the retrieved context.
- * Include source references or section names.
- * State uncertainty when the source material does not fully answer the question.

Compare one grounded answer with an ungrounded answer where the model is asked to answer without the source material.

Daily Deliverables

- * A written RAG flow diagram or step-by-step explanation.
- * Three sample questions.
- * Retrieved context for each question.
- * Final grounded answers with source references.
- * A short critique of retrieval quality: what worked, what failed, and what could improve.

Self-Check

You should be able to answer:

- * What problem does RAG solve?
- * Why does chunking matter?
- * What makes a retrieved passage relevant?
- * Why should an answer cite its sources?
- * What should happen when the retrieved context is incomplete?

Day 4: Context Engineering And Workflow Design

Goal

Learn how to package the right information for multi-step AI workflows.

Learn

Study these concepts:

- * What context means in an AI application.
- * Context windows and context limits.
- * Selecting, summarizing, compressing, and ordering context.
- * Short-term conversation memory vs long-term stored memory.
- * Including user preferences, retrieved knowledge, tool results, and task state.
- * Avoiding noisy, stale, sensitive, or irrelevant context.
- * Why information can be missed when buried in long context.

Public learning material:

- * Lost in the Middle paper: <https://arxiv.org/abs/2307.03172>
- * OpenAI Agents Guide: <https://platform.openai.com/docs/guides/agents>
- * OpenAI Agent Evals Guide: <https://platform.openai.com/docs/guides/agent-evals>

Exercise

Choose one workflow:

- * Support ticket triage.
- * HR policy Q&A.
- * Product documentation assistant.
- * Meeting notes assistant.
- * Data question assistant.

Design a context package for the workflow. Include:

- * The incoming user request.
- * Relevant prior history.
- * Retrieved documents, rules, or definitions.
- * Output requirements.
- * Human review point.
- * Information to exclude, summarize, or cite.

Create two versions:

- * A noisy version with too much or poorly organized context.
- * A cleaner version that prioritizes what the model needs.

Explain why the cleaner version should perform better.

Daily Deliverables

- * A context template for your chosen workflow.
- * A list of what to include, summarize, exclude, and cite.
- * A short writeup of likely failure modes and controls.

Self-Check

You should be able to answer:

- * What context is essential for this workflow?
- * What context is distracting or risky?
- * What information should be summarized instead of pasted in full?
- * Where should human review happen?
- * How could poor context cause a wrong answer?

Day 5: Mini Capstone And Presentation

Goal

Combine prompting, API calls, RAG, and context engineering into a small assistant design.

Capstone Options

Choose one:

- * HR policy Q&A assistant.
- * Product documentation assistant.
- * Meeting notes Q&A assistant.
- * Support ticket triage assistant.

Recommended default: HR policy Q&A assistant, because it naturally combines retrieval, source-grounded answers, uncertainty, and human review.

Build Or Design

Define the workflow using this loop:

- * Trigger: what starts the workflow?
- * Understand: what does the assistant need to identify?
- * Gather context: what documents, data, or history does it need?
- * Reason: what rules or constraints should guide the response?
- * Prepare output: what should the assistant produce?
- * Human checkpoint: where should a person review or approve?
- * Action or update: what happens after review?

- * Monitor and learn: how would quality improve over time?

Create either:

- * A small working prototype.
- * A detailed design with pseudocode, prompts, example inputs, and example outputs.

Presentation

Prepare a 10-minute walkthrough covering:

- * Problem statement.
- * User workflow.
- * Prompt design.
- * API flow or pseudocode.
- * Retrieval and context strategy.
- * Example input and output.
- * Human review point.
- * Risks and controls.
- * Evaluation plan.
- * Next improvements.

Daily Deliverables

- * Final short report or slide-style writeup.
- * Prompt design.
- * API flow or pseudocode.
- * Retrieval and context strategy.
- * Example input and output.
- * Risks, human checkpoints, and evaluation plan.
- * 10-minute demo or presentation.

Self-Check

You should be able to answer:

- * What does the assistant know?
- * What does it not know?
- * What context does it use?
- * Where can it fail?
- * Where does a human review the output?
- * How would you measure whether it helped?

Final Rubric

Use this rubric to evaluate your own work at the end of the week.

Area	Strong Evidence
Prompting	You can write clear instructions, constraints, examples, and output formats
API usage	You can make or clearly describe a basic call, parse output, and handle simple failures
RAG understanding	You can explain retrieval, grounding, citations, and limitations
Context engineering	You can choose useful context and avoid unnecessary noise
Agent readiness	You can map a workflow into trigger, context, reasoning, human review, and action
Communication	You can explain design choices, risks, and next steps clearly

Final Submission Checklist

At the end of the week, prepare:

- * Day 1 prompt notes and before/after examples.
- * Day 2 API or pseudocode exercise.
- * Day 3 RAG questions, retrieved context, answers, and critique.
- * Day 4 context template and failure-mode notes.
- * Day 5 capstone report or presentation.

Boundaries

- * Keep the capstone small enough to explain end to end in 10 minutes.
- * Do not treat model generated output/recommendations as final decisions.
- * Include human review for risky, external, customer-facing, legal, financial, HR, or compliance-related outputs.
- * Prefer clear and inspectable workflows over complex automation.
- * When the model is uncertain or context is incomplete, the output should not be treated as reliable.